Unit-III

3.2 Anomalies in Relational Model

Anomalies in the relational model refer to inconsistencies or errors that can arise when working with relational databases, specifically in the context of data insertion, deletion and modification. Anomalies can compromise data integrity and make database management inefficient.

How Are Anomalies Caused in DBMS?

Anomalies in DBMS are caused by poor management of storing everything in the flat database, lack of normalization, data redundancy and improper use of primary or foreign keys. These issues result in inconsistencies during insert, update or delete operations, leading to data integrity problems. The three primary types of anomalies are:

- **Insertion Anomalies:** These anomalies occur when it is not possible to insert data into a database because the required fields are missing or because the data is incomplete. For example, if a database requires that every record has a primary key, but no value is provided for a particular record, it cannot be inserted into the database.
- **Deletion anomalies:** These anomalies occur when deleting a record from a database and can result in the unintentional loss of data. For example, if a database contains information about customers and orders, deleting a customer record may also delete all the orders associated with that customer.
- **Update anomalies:** These anomalies occur when modifying data in a database and can result in inconsistencies or errors. For example, if a database contains information about employees and their salaries, updating an employee's salary in one record but not in all related records could lead to incorrect calculations and reporting.

STUDENT Table:

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD-COUNTRY	STUD_AGE
1	RAM	9716271721	Haryana	India	20
2	RAM	9898291281	Punjab	India	19

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD-COUNTRY	STUD_AGE
3	SUJIT	7898291981	Rajasthan	India	18
4	SURESH		Punjab	India	21

STUDENT_COURSE:

STUD_NO	COURSE_NO	COURSE_NAME
1	C1	DBMS
2	C2	Computer Networks
1	C2	Computer Networks

1. Insertion Anomaly

If a tuple is inserted in referencing relation and referencing attribute value is not present in referenced attribute, it will not allow insertion in referencing relation. In simpler words, an insertion anomaly occurs when adding a new row to a table leads to inconsistencies.

Example: If we try to insert a record into the STUDENT_COURSE table with STUD_NO = 7, it will not be allowed because there is no corresponding STUD_NO = 7 in the STUDENT table.

2. Deletion and Updation Anomaly:

If a tuple is deleted or updated from referenced relation and the referenced attribute value is used by referencing attribute in referencing relation, it will not allow deleting the tuple from referenced relation.

Example: If we want to update a record from STUDENT_COURSE with STUD_NO =1, We have to update it in both rows of the table. If we try to delete a record from the STUDENT table with STUD_NO = 1, it will not be allowed because there are corresponding records in the STUDENT_COURSE table referencing STUD_NO = 1. To avoid this, the following can be used in query:

- **ON DELETE/UPDATE SET NULL:** If a tuple is deleted or updated from referenced relation and the referenced attribute value is used by referencing attribute in referencing relation, it will delete/update the tuple from referenced relation and set the value of referencing attribute to NULL.
- **ON DELETE/UPDATE CASCADE:** If a tuple is deleted or updated from referenced relation and the referenced attribute value is used by referencing attribute in referencing relation, it will delete/update the tuple from referenced relation and referencing relation as well.

Removal of Anomalies

Anomalies in DBMS can be removed by applying normalization. Normalization involves organizing data into tables and applying rules to ensure data is stored in a consistent and efficient manner. By reducing data redundancy and ensuring data integrity, normalization helps to eliminate anomalies and improve the overall quality of the database.

According to **E. F. Codd**, who is the inventor of the Relational Database, the goals of Normalization include:

- It helps in vacating all the repeated data from the database.
- It helps in removing undesirable deletion, insertion and update anomalies.
- It helps in making a proper and useful relationship between tables.

3.3 Inference Rules in DBMS

Inference rules in databases are also known as **Armstrong's Axioms in Functional Dependency.** These rules govern the functional dependencies in a relational database. From inference rules a new functional dependency can be derived using other FDs. These rules were introduced by **William W. Armstrong**. In this article, we will come to know about all the rules proposed by him. Also, we will be exploring the prerequisites for it and will understand the topic in a better way.

Prerequisites

- **Attributes:** When we talk about databases, we think of them as organized collections of information. Imagine that you have a table called "Student." Now, this table has columns, which we also call "Attributes." These columns define specific details about the students. For example:
    - **Student_name:** This column stores the names of the students.
    - **Roll_no:** Here, we keep track of their roll numbers.

    - **Marks:** And finally, we record their exam scores.

- **Functional Dependencies (FDs)** are like the building blocks of a database. Imagine you have a bunch of attributes (think of them as characteristics) in a table. These attributes can be related to each other in interesting ways or say logically. For example, Roll_no → Marks means that from Roll_no we can get the Marks of the student, which shows that they are Roll_no is logically related to Marks.

Inference Rules

There are 6 inference rules, which are defined below:

- **Reflexive Rule:** According to this rule, if B is a subset of A then A logically determines B. Formally, **B ⊆ A** then **A → B**.
    - Example: Let us take an example of the Address (A) of a house, which contains so many parameters like House no, Street no, City etc. These all are the subsets of A. Thus, address (A) → House no. (B).
- **Augmentation Rule:** It is also known as **Partial dependency**. According to this rule, If A logically determines B, then adding any extra attribute doesn't change the basic functional dependency.
    - Example: **A → B**, then adding any extra attribute let say C will give **AC → BC** and doesn't make any change.
- **Transitive rule:** Transitive rule states that if A determines B and B determines C, then it can be said that A indirectly determines B.
    - Example: If **A → B** and **B → C** then **A → C**.
- **Union Rule:** Union rule states that If A determines B and C, then A determines BC.
    - Example: If **A → B** and **A → C** then **A → BC.**
- **Decomposition Rule:** It is perfectly reverse of the above Union rule. According to this rule, If A determined BC then it can be decomposed as A → B and A → C.
    - Example: If **A → BC** then **A → B** and **A → C.**
- **Pseudo Transitive Rule:** According to this rule, If A determined B and BC determines D then BC determines D.
    - Example: If **A → B** and **BC → D** then **AC → D**.

3.4 Properties of Relational Decomposition

When a relation in the relational model is not appropriate normal form then the decomposition of a relation is required. In a database, breaking down the table into multiple tables termed as decomposition. The properties of a relational decomposition are listed below :

1. **Attribute Preservation:** Using functional dependencies the algorithms decompose the universal relation schema R in a set of relation schemas D = { R1, R2, ... Rn } relational database schema, where 'D' is called the Decomposition of R. The attributes in R will appear in at least one relation schema Ri in the decomposition, i.e., no attribute is lost. This is called the *Attribute Preservation* condition of decomposition.
2. **Dependency Preservation:** If each functional dependency X->Y specified in F appears directly in one of the relation schemas Ri in the decomposition D or could be inferred from the dependencies that appear in some Ri. This is the *Dependency Preservation*. If a decomposition is not dependency preserving some dependency is lost in decomposition. To check this condition, take the JOIN of 2 or more relations in the decomposition. For example:
3. R = (A, B, C)
4. F = {A ->B, B->C}
5. Key = {A}
6.
7. R is not in BCNF.

Decomposition R1 = (A, B), R2 = (B, C)

R1 and R2 are in BCNF, Lossless-join decomposition, Dependency preserving. Each Functional Dependency specified in F either appears directly in one of the relations in the decomposition. It is not necessary that all dependencies from the relation R appear in some relation Ri. It is sufficient that the union of the dependencies on all the relations Ri be equivalent to the dependencies on R.

8. **Non Additive Join Property:** Another property of decomposition is that D should possess is the *Non Additive Join Property*, which ensures that no spurious tuples are generated when a NATURAL JOIN operation is applied to the relations resulting from the decomposition.

9. **No redundancy:** Decomposition is used to eliminate some of the problems of bad design like anomalies, inconsistencies, and redundancy.If the relation has no proper decomposition, then it may lead to problems like loss of information.
10. **Lossless Join:** Lossless join property is a feature of decomposition supported by normalization. It is the ability to ensure that any instance of the original relation can be identified from corresponding instances in the smaller relations. For example: R : relation, F : set of functional dependencies on R, X, Y : decomposition of R, A decomposition {R1, R2, …, Rn} of a relation R is called a lossless decomposition for R if the natural join of R1, R2, …, Rn produces exactly the relation R. A decomposition is lossless if we can recover: R(A, B, C) -> Decompose -> R1(A, B) R2(A, C) -> Recover -> R'(A, B, C) Thus, R' = R Decomposition is lossless if: X intersection Y -> X, that is: all attributes common to both X and Y functionally determine ALL the attributes in X. X intersection Y -> Y, that is: all attributes common to both X and Y functionally determine ALL the attributes in Y If X intersection Y forms a superkey of either X or Y, the decomposition of R is a lossless decomposition.